

计算机系统导论下

学习笔记

内容提炼

2026 年 9 月 23 日

说明

本笔记对应卡内基梅隆大学 15-213 所用教材 *Computer Systems: A Programmer's Perspective* (Bryant & O'Hallaron, 第三版) 的第二部分与第三部分, 即第 7–12 章。中文章名、节名与子节名按机械工业出版社《深入理解计算机系统》第三版译名, 层级与顺序与英文原书一致。试作版充实第 7 章已课堂讲授的内容 (至库打桩; 第 7.12、7.14 节仍待补), 并在第 7.15 节小结之后收录课堂「练习」(练习题 7.9、家庭作业 7.15 与上机测量); 第 8 章按讲义 14-ecf-procs 充实异常、进程与进程控制 (第 8.4.4 节休眠以及第 8.5–8.7 节仍待补); 第 9–12 章保留原书标题。正文以公式陈述定义、成立条件、关键推导与结论。每一公式后写明: 数学上如何由输入算出输出, 以及得到的数在问题中的实际含义。已充实章节中有信息量的示意图随文插入, 图号沿用原书, 并在图下注释符号。

目录

说明	i
第七章 链接	1
7.1 编译器驱动程序	1
7.2 静态链接	2
7.3 目标文件	2
7.4 可重定位目标文件	3
7.5 符号和符号表	4
7.6 符号解析	4
7.7 重定位	7
7.8 可执行目标文件	9
7.9 加载可执行目标文件	9
7.10 动态链接共享库	10
7.11 从应用程序中加载和链接共享库	11
7.12 位置无关代码	11
7.13 库打桩机制	11
7.14 处理目标文件的工具	12
7.15 小结	13
7.16 练习	13
第八章 异常控制流	16
8.1 异常	16
8.2 进程	20
8.3 系统调用错误处理	23

8.4	进程控制	24
8.5	信号	28
8.6	非本地跳转	30
8.7	操作进程的工具	30
8.8	小结	30
第九章	虚拟内存	31
9.1	物理和虚拟寻址	31
9.2	地址空间	31
9.3	虚拟内存作为缓存的工具	31
9.4	虚拟内存作为内存管理的工具	32
9.5	虚拟内存作为内存保护的工具	32
9.6	地址翻译	32
9.7	案例研究: Intel Core i7/Linux 内存系统	33
9.8	内存映射	34
9.9	动态内存分配	34
9.10	垃圾收集	36
9.11	C 程序中常见的与内存有关的错误	37
9.12	小结	39
第十章	系统级 I/O	40
10.1	Unix I/O	40
10.2	文件	40
10.3	打开和关闭文件	40
10.4	读和写文件	40
10.5	用 Rio 包健壮地读写	40
10.6	读取文件元数据	41
10.7	读取目录内容	41
10.8	共享文件	41
10.9	I/O 重定向	41
10.10	标准 I/O	41
10.11	综合: 该使用哪些 I/O 函数	42

10.12 小结	42
第十一章 网络编程	43
11.1 客户端-服务器编程模型	43
11.2 网络	43
11.3 全球 IP 因特网	43
11.4 套接字接口	44
11.5 Web 服务器	45
11.6 综合: TINY Web 服务器	46
11.7 小结	46
第十二章 并发编程	47
12.1 基于进程的并发编程	47
12.2 基于 I/O 多路复用的并发编程	47
12.3 基于线程的并发编程	48
12.4 多线程程序中的共享变量	49
12.5 用信号量同步线程	50
12.6 使用线程提高并行性	50
12.7 其他并发问题	51
12.8 小结	51

第七章 链接

链接（linking）把多个代码与数据片段收集并组合成一个可被加载到内存并执行的文件。它可发生在编译时（compile time）、加载时（load time）或运行时（runtime）。静态链接器（linker）ld 在生成可执行文件时完成前一种；动态链接器在后两种完成。

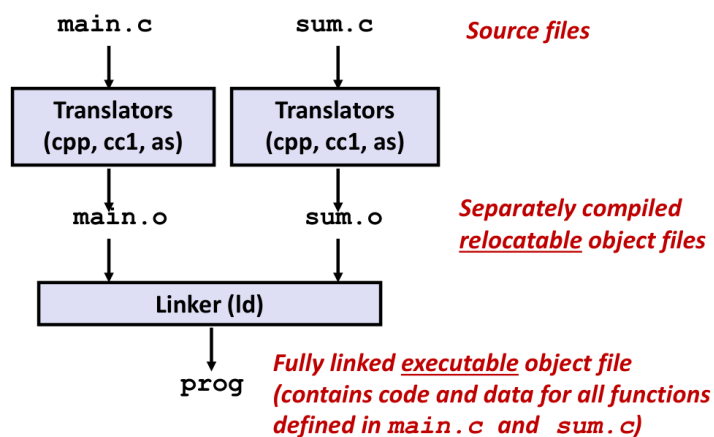
7.1 编译器驱动程序

Carnegie Mellon

Linking

- Programs are translated and linked using a *compiler driver*:

- `linux> gcc -Og -o prog main.c sum.c`
- `linux> ./prog`



Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

5

图 7.2 编译器驱动程序调用预处理器、编译器、汇编器与链接器。

符号：main.c/sum.c：源文件；cpp,cc1,as,ld：各阶段；prog：可执行文件

贯穿本章的示例由 main.c 与 sum.c 组成：前者定义全局数组 array 并调用 sum，后者定义求和函数。gcc 是编译器驱动程序（compiler driver），按需调用预处理器 cpp、编译器 cc1、汇编器 as 与链接器 ld。

```
linux> gcc -Og -o prog main.c sum.c
linux> ./prog
```

翻译链写成

$$\text{main.c} \xrightarrow{\text{cpp}} \text{main.i} \xrightarrow{\text{cc1}} \text{main.s} \xrightarrow{\text{as}} \text{main.o},$$

对 `sum.c` 同样得到 `sum.o`，最后

$$(\text{main.o}, \text{sum.o}, \text{系统目标文件}) \xrightarrow{\text{ld}} \text{prog}.$$

每个箭头的输入是上一阶段的 ASCII 或二进制文件，输出是下一阶段的文件；`prog` 是可执行目标文件，不是源文件。`-Og` 只改变优化程度，不改变上述阶段划分。

运行 `./prog` 时，shell 调用加载器（loader）把 `prog` 映射进内存并把控制转到入口。加载器的输入是磁盘上的 ELF 可执行文件，输出是进程的虚拟地址空间中一段可执行映像。

7.2 静态链接

静态链接（static linking）指在生成 `prog` 时就把所需目标模块的代码与数据拷入可执行文件，运行时不再依赖那些 `.o` 或静态库 `.a`。它不是「运行时指令不能改」，也不是把操作系统、堆、栈一并拷进 `prog`。

链接器对输入的可重定位目标文件做两步。符号解析（symbol resolution）把每个引用绑定到恰好一个定义：

$$\text{Resolve}(r) = d, \quad r \in R, d \in D, |\{d\}| = 1.$$

输入是引用集合 R 与定义集合 D ；输出是从引用到定义的函数。 $\text{Resolve}(r)$ 表示「名字 r 最终用哪一份实体」，不是内存地址。找不到或强符号冲突则链接失败。

重定位（relocation）在合并节之后，把相对位置改成最终虚拟地址，并改写引用处的字节。若符号 s 被分配到运行时地址 $\text{ADDR}(s)$ ，则指令或数据中的占位字段被换成由 $\text{ADDR}(s)$ 算出的立即数或位移。 $\text{ADDR}(s)$ 是虚拟地址（无量纲的字节编号），取值在用户空间可映射范围内，例如本讲示例中代码约从 `0x400000` 起。

分开编译的成立条件：只改一个 `.c` 时，重编该文件得到新 `.o`，再与未改的 `.o` 重新链接即可，不必重编全部源文件。

7.3 目标文件

链接器处理三类目标文件（object file）。可重定位目标文件 `.o` 由恰好一个 `.c` 汇编得到，地址未定死，用于再与其它模块合并。可执行目标文件（传统名 `a.out`）含可直接

拷入内存执行的代码与数据。共享目标文件 `.so` 是可在加载时或运行时动态链接的特殊可重定位文件，Windows 上称为 DLL。

7.4 可重定位目标文件

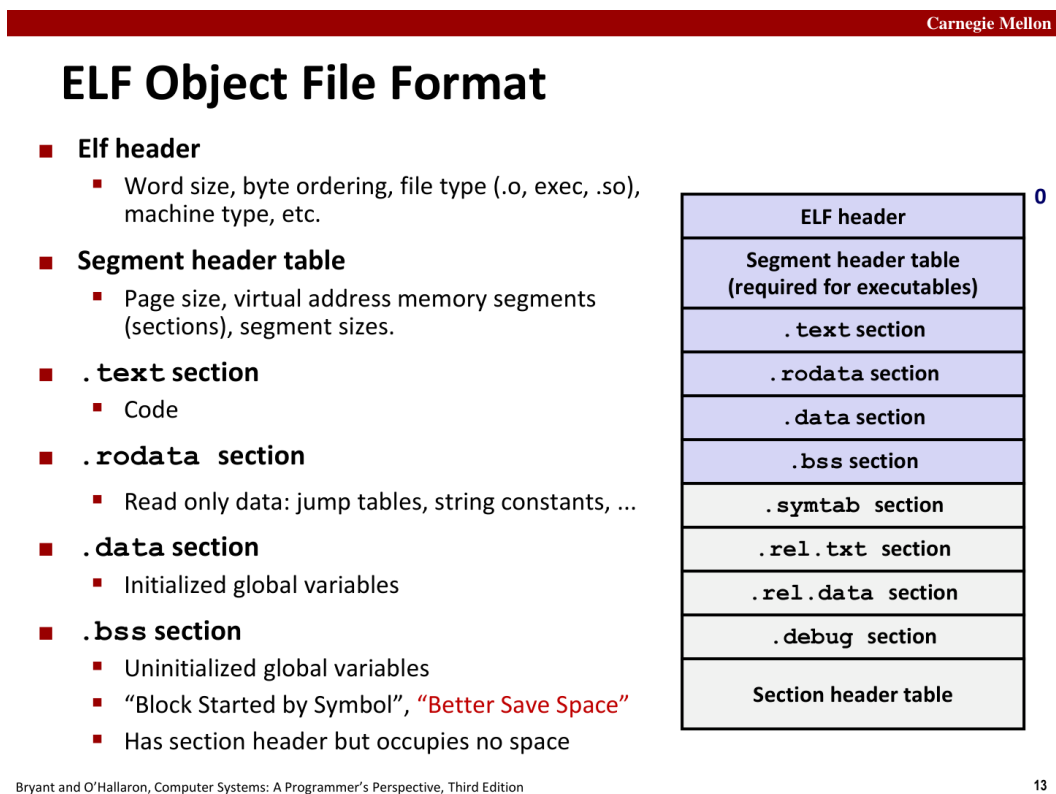


图 7.4 ELF 可重定位目标文件的节布局。

符号：ELF header：类型与机器；`.text/.rodata/.data/.bss`：代码与数据；`.symtab`：符号表；section header table：节索引

Linux 上三类目标文件共用 ELF (Executable and Linkable Format)。文件从偏移 0 起依次为 ELF 头、(可执行文件需要的)段头表、各节、节头表。

节头表给出每个节在文件中的偏移与大小，供链接器按节合并。节标志(如是否含指令、是否可写、是否占用运行时内存)是给链接器的分类标签，用来决定哪些节打进同一加载段。段头表给出加载时 `mmap` 的虚拟地址、长度与页权限 `r/w/x`；进程真正生效的内存保护看段标志，不看节标志。同一段文件字节可以被两套索引描述：节头说「这是 `.text`」，段头说「映射到虚拟地址 `V`、权限 `r-x`」。

常用节：`.text` 为机器指令；`.rodata` 为只读数据；`.data` 为已初始化全局/静态变量；`.bss` 为未初始化或按零初始化的全局/静态变量。若未初始化全局对象的运行时长度为 n 字节，则

$$|.bss|_{\text{file}} \approx 0, \quad |.bss|_{\text{mem}} = n.$$

左边是磁盘占用（节头记下大小，不存 n 个零），右边是加载后在数据段中留出的字节数，内容为 0。 n 是字节计数，不是变量的值。

`.symtab` 是符号表：函数名、全局/静态变量名，以及该符号所在节与节内偏移。它回答「名为 `sum` 的对象在哪一节、多大」，不能替代节头表；ELF 头先指向节头表，节头表再指出 `.symtab` 的文件偏移。`.rel.text` 与 `.rel.data`（或带加数的 `.rela.*`）分别记录代码中的指令与数据中的指针哪些字节需要在链接时改写。`.debug` 在 `gcc -g` 时生成，供调试，不参与取指。

7.5 符号和符号表

对模块 m （一个 `.o`），链接符号分三类。全局符号由 m 定义且可被其它模块引用，对应无 `static` 的函数与全局变量。外部符号是 m 引用、定义在其它模块的全局符号。局部链接符号仅由 m 定义与引用，对应带 `static` 的函数与全局变量。局部链接符号不是函数内自动变量：`main` 里的 `val`、`sum` 里的 `i` 与 `s` 在栈上，链接器不处理它们。

函数内 `static` 变量存储在 `.data` 或 `.bss`，进程内一份，函数返回后仍在。若 f 与 g 各有 `static int x`，编译器为每个定义分配独立存储，并在符号表中写成互异局部名（如 `x.1`、`x.2`）。这些名字对其它 `.o` 不可见。

在示例中，`main.o` 定义 `main`、`array` 并引用 `sum`；`sum.o` 定义 `sum`。解析把前者对 `sum` 的引用接到后者的定义。

7.6 符号解析

解析把每个引用与输入模块符号表中恰好一个定义关联。模块内局部符号由编译器保证唯一；全局符号可能跨模块重名，须按下一小节的强弱规则处理。若引用在全部输入中都无法定义，链接器报 `undefined reference` 并终止。

7.6.1 链接器如何解析多重定义的全局符号

编译器把每个全局符号标为强（strong）或弱（weak），汇编器写入符号表。函数与已初始化全局变量是强符号；未初始化全局变量（如文件作用域 `int foo;`）是弱符号。`int foo=0;` 有初值，是强符号。

Linux 链接器规则：不允许多个同名强符号，否则报错；一个强符号与若干弱符号同名则选强符号，弱引用接到该定义；多个弱符号同名则任意选一个（可用 `gcc -fno-common` 改为遇多重定义即报错）。

若模块 1 有强符号 x 而模块 2 有弱符号 x ，则

$$\text{REF}(x.2) \rightarrow \text{DEF}(x.1).$$

箭头左边是引用所在模块，右边是被选中的定义所在模块。输出不是「两个变量」，而是全程序共用一份 x 。

若一边是 `int x; int y;`、另一边是 `double x;`，弱符号合并后两边对 $|x|$ 的看法不同： $x86-64$ 上 $|\text{int}| = 4$ 、 $|\text{double}| = 8$ ，对 x 的 8 字节写可能覆盖相邻的 y 。若 x 、 y 已初始化而为强符号，布局已定，对接到 `int x` 上的 `double` 写入会覆盖 y 。

实践：能不用全局则不用；仅本文件用则加 `static`；跨文件定义时初始化以得到强符号；引用别处的定义用 `extern`。

7.6.2 与静态库链接

Carnegie Mellon

Linking with Static Libraries

```
#include <stdio.h>
#include "vector.h"

int x[2] = {1, 2};
int y[2] = {3, 4};
int z[2];

int main(int argc, char** argv)
{
    addvec(x, y, z, 2);
    printf("z = [%d %d]\n",
           z[0], z[1]);
    return 0;          main2.c
}
```

```
libvector.a

void addvec(int *x, int *y,
            int *z, int n) {
    int i;

    for (i = 0; i < n; i++)
        z[i] = x[i] + y[i];
}                                addvec.c

void multvec(int *x, int *y,
             int *z, int n)
{
    int i;

    for (i = 0; i < n; i++)
        z[i] = x[i] * y[i];
}                                multvec.c
```

Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

36

图 7.6 归档器把多个 .o 打成静态库 .a。

符号：ar：归档器；libc.a：静态库；成员：被按需抽出的 .o

静态库（static library）是带索引的归档（archive）.a，由 ar 把许多 .o 打包而成。链接器只把当前未解析引用所需要的那些成员拷进可执行文件，不是把整个归档拷进去。生成 prog 之后运行不再需要该 .a；重新链接时仍需提供 .a。

不用库时的两种极端都不合适：把全部标准函数放进一个巨大 .o 会浪费空间与重编

时间；每个函数一个.o 则要求程序员显式列出依赖。归档把「一函数一.o」装进一个文件，由链接器按符号抽取。

课堂数据（2015 年演示系统）：libc.a 约 4.6 MB、1496 个成员；libm.a 约 2 MB、444 个成员。这些数是归档体积与成员个数，不是某个函数的运行时间。

向量库示例：addvec.c 与 multivec.c 经

```
ar rcs libvector.a addvec.o multivec.o
```

打成 libvector.a。main2.c 调用 addvec 与 printf。#include <stdio.h> 按系统路径找头文件；#include "vector.h" 通常先搜当前目录。头文件只提供声明，不把库代码链进程序。链接时抽出 addvec.o 以及 libc.a 中的 printf.o 及其依赖；不抽 multivec.o。可执行文件常命名为 prog2c，其中 c 表示 compile-time 静态链接。

7.6.3 链接器如何使用静态库来解析引用

Carnegie Mellon

Using Static Libraries

■ Linker's algorithm for resolving external references:

- Scan .o files and .a files in the command line order.
- During the scan, keep a list of the current unresolved references.
- As each new .o or .a file, *obj*, is encountered, try to resolve each unresolved reference in the list against the symbols defined in *obj*.
- If any entries in the unresolved list at end of scan, then error.

■ Problem:

- Command line order matters!
- Moral: put libraries at the end of the command line.

```
unix> gcc -static -o prog2c -L. -lvector main2.o
main2.o: In function `main':
main2.c(.text+0x19): undefined reference to `addvec'
collect2: error: ld returned 1 exit status
```

Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

38

图 7.7 与静态库链接时只抽出解析当前未定义引用所需的成员。

符号：main2.o：程序模块；libvector.a/libc.a：静态库；*U*：尚未定义的符号集

从左到右扫描命令行上的.o 与.a。维护三个集合，初值皆空：*E* 为将并入可执行文件的目标文件集合，*U* 为尚未定义的引用符号集合，*D* 为已经见到定义的符号集合。

对每个输入 f : 若 f 是目标文件, 则

$$E \leftarrow E \cup \{f\}, \quad U \leftarrow (U \cup \text{Ref}(f)) \setminus \text{Def}(f), \quad D \leftarrow D \cup \text{Def}(f).$$

$\text{Ref}(f)$ 、 $\text{Def}(f)$ 分别是 f 中的引用名与定义名。 U 中剩下的是「还没有定义的名字」, 不是地址。

若 f 是归档, 则反复查找能定义 U 中某个符号的成员 m , 把 m 当作目标文件加入 E 并更新 U 、 D , 直到不再变化; 未被选中的成员丢弃。扫描结束时若 $U \neq \emptyset$, 报错。

因此库只解析「当时 U 里已有的名字」。gcc -L. libtest.o -lmine 先让 libfun 进入 U , 再扫 libmine.a 抽出定义; gcc -L. -lmine libtest.o 扫库时 $U = \emptyset$, 一个成员都不抽, 随后出现的引用无法解析。准则: 把库放在命令行末尾; 若 A 依赖 B , 写 -lA -lB; 循环依赖可重复写某个库。

共享库 .so 不按成员抽取拷贝, 链接时通常只记录依赖; 但 GNU 链接器在 --as-needed 下仍按从左到右消当前未解析符号, 故 -l 仍宜放在程序 .o 之后。运行时需要能找到 .so, 这与静态 .a 不同。

7.7 重定位

Carnegie Mellon

Relocation Entries

```
int array[2] = {1, 2};

int main(int argc, char** argv)
{
    int val = sum(array, 2);
    return val;
}
```

main.c

```
0000000000000000 <main>:
0:  48 83 ec 08          sub    $0x8,%rsp
4:  be 02 00 00 00      mov    $0x2,%esi
9:  bf 00 00 00 00      mov    $0x0,%edi          # %edi = &array
                          a: R_X86_64_32 array      # Relocation entry

e:  e8 00 00 00 00      callq 13 <main+0x13>      # sum()
                          f: R_X86_64_PC32 sum-0x4  # Relocation entry
13: 48 83 c4 08          add    $0x8,%rsp
17: c3                   retq

main.o
```

Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

Source: objdump -r -d main.o 27

图 7.8 重定位把各模块的代码与数据节合并到可执行文件。

符号: main.o/sum.o: 可重定位模块; .text/.data: 被合并的节; 最终地址: ADDR(s)

符号解析完成后，链接器合并各 .o 的同类节，为每个符号选定 $\text{ADDR}(s)$ ，再按重定位条目改写引用。

7.7.1 重定位条目

汇编器遇到最终位置未知的对象时生成重定位条目，代码条目在 `.rel.text`，数据指针条目在 `.rel.data`。ELF 条目含偏移 $r.offset$ 、符号 $r.symbol$ 、类型 $r.type$ 与加数 $r.addend$ 。

对 `main.o` 中 `sum(array,2)`，`objdump -r -d` 可见两处占位零。`mov $0,%edi` 对应类型 `R_X86_64_32`、符号 `array`：把 `array` 的 32 位绝对地址写入立即数。`call` 的 `e8 00 00 00 00` 对应类型 `R_X86_64_PC32`、符号 `sum`、加数 `-4`：写入相对下一条指令的 32 位位移。常数 2 编 `main.o` 时已确定，无重定位条目。

7.7.2 重定位符号引用

链接器已选定节地址 $\text{ADDR}(s)$ 与符号地址 $\text{ADDR}(r.symbol)$ 。对节 s 中每条条目 r ，令 $refptr = s + r.offset$ 指向待改的 4 字节。

PC 相对 (`R_X86_64_PC32`):

$$\begin{aligned} refaddr &= \text{ADDR}(s) + r.offset, \\ *refptr &= \text{ADDR}(r.symbol) + r.addend - refaddr. \end{aligned}$$

$refaddr$ 是这 4 字节字段自己的运行时地址； $*refptr$ 是写入指令的有符号位移，单位字节，使 CPU 用「下一条指令地址加位移」落到目标。

示例： $\text{ADDR}(.text) = 0x4004d0$ ， $r.offset = 0xf$ ， $\text{ADDR}(\text{sum}) = 0x4004e8$ ， $r.addend = -4$ ，则

$$refaddr = 0x4004df, \quad *refptr = 0x5.$$

`0x5` 不是 `sum` 的绝对地址。执行 `call` 时 PC 为下一条指令地址 `0x4004e3`，

$$0x4004e3 + 0x5 = 0x4004e8 = \text{ADDR}(\text{sum}).$$

绝对寻址 (`R_X86_64_32`):

$$*refptr = \text{ADDR}(r.symbol) + r.addend.$$

对 `array`， $r.addend = 0$ ， $\text{ADDR}(\text{array}) = 0x601018$ ，立即数即该虚拟地址，送入 `%edi` 作为指针实参。

7.8 可执行目标文件

可执行 ELF 含 ELF 头、段头表、已合并的 `.init/.text/.rodata/.data`，以及运行时展开的 `.bss`。`.symtab` 与 `.debug` 可保留供调试，加载执行不必依赖它们。静态链接后 `main` 与 `sum` 出现在同一 `.text` 中，带最终虚拟地址。

7.9 加载可执行目标文件

Carnegie Mellon

Quiz Time!

Canvas Quiz: Day 16 – Linking

Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

30

图 7.11 加载可执行文件后的典型虚拟内存布局。

符号：0x400000：只读代码段起点；`.data/.bss`：数据段；`heap/stack`：堆与用户栈

加载器按段头 `mmap` 可执行文件。典型布局（本讲示例）：只读可执行段从 0x400000 起，含 `.init`、`.text`、`.rodata`；其后为可读可写数据段，含 `.data` 与清零后的 `.bss`；再往上是堆（`brk`）、共享库映射区、向下增长的用户栈（`%rsp`），最高处为用户不可见的内核虚拟内存。

未初始化的全局/`static` 变量在加载之后、`main` 之前被置 0（`.bss`）。函数内非 `static` 局部变量在栈上，加载不清零，使用前必须自行赋值。

7.10 动态链接共享库

静态库把成员 `.o` 在生成可执行文件时拷进去，带来三处浪费：磁盘上每个程序各带一份 `libc` 代码；每个进程映像再各有一份；修库后每个应用必须重新链接。共享库（shared library，Linux 上为 `.so`，Windows 上为 DLL）把代码留在一份库文件里，在加载时或运行时再链进进程。

Dynamic Linking at Load-time

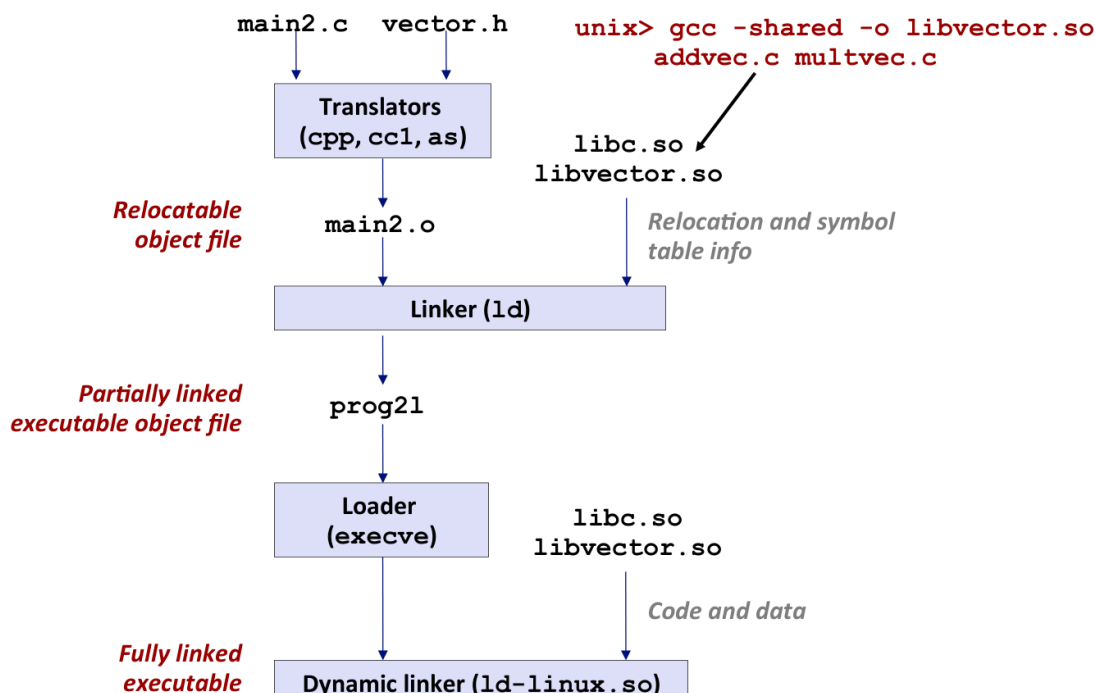


图 7.16 加载时动态链接：磁盘上的可执行文件只部分链接，进内存后由动态链接器焊上共享库。

符号：prog21：部分链接的可执行文件；ld：静态链接器，只记录对 `.so` 的依赖；ld-linux.so：动态链接器；libc.so/libvector.so：共享库

令 P 为磁盘上的可执行文件， $L_1, \dots, L_k$ 为其动态依赖。加载时链接的顺序是

$$\text{mmap}(P) \rightarrow \text{mmap}(\text{ld-linux.so}) \rightarrow \text{mmap}(L_i) \rightarrow \text{Reloc}_{\text{dyn}} \rightarrow \text{main}.$$

前三个 `mmap` 的输入是文件，输出是进程虚拟地址空间中的映射； $\text{Reloc}_{\text{dyn}}$ 读 P 与各 L_i 的动态符号表（`.dynsym`）及动态重定位节（如 `.rela.plt`、`.rela.dyn`），把未填地址写成 `ADDR(s)`。`ADDR(s)` 仍是虚拟地址；填完后这份进程内存才是 fully linked， P 在磁盘上始终只记依赖、不拷库代码。

与静态 `.rel.text/.rel.data` 的分工：后者由 ld 在生成 P 时用掉；对 `printf`、`addvec` 等库符号，加载时改的是 P 与 `.so` 里留给动态链接器的重定位表。部分符号可惰性绑定：第一次调用才填 GOT/PLT 中对应项，不是进 `main` 前全部填完。

共享库由

```
gcc -shared -o libvector.so addvec.c multvec.c
```

生成。链接 prog21 时 addvec 的机器码不进可执行文件，只写入需要 libvector.so 的记录。

7.11 从应用程序中加载和链接共享库

运行时链接在已经进入 main 之后由程序调用 dlopen 完成，而不是启动时由 ld-linux.so 根据 DT_NEEDED 自动打开。

$$h = \text{dlopen}(p, f), \quad a = \text{dlsym}(h, n), \quad \text{dlclose}(h).$$

路径 p 是 .so 文件名， f 为标志（如 RTLD_LAZY 表示用到符号再解析）；输出 h 是句柄（失败为 NULL）。 n 是以空字符结尾的符号名字符串；输出 a 是该符号在本次映射中的地址，不是链接时填进指令的立即数。dlerror 在失败时给出错误串。

编含 dlopen 的程序时，可执行文件含 main、数据对象、库路径与函数名字符串，以及函数指针变量；不含函数 addvec 的代码，也没有对它的链接引用。addvec 的定义与动态符号在编 libvector.so 时已进该库的 .dynsym。因此该程序的 gcc 命令行不必出现 libvector.so（通常只需 -ldl）。dlopen 不会根据后面会不会调用而自行决定；执行到那一行才映射库，再由 dlsym 按字符串查找。

对该路径，可执行文件不为 addvec 生成 .rel 条目；被 dlopen 打开的 .so 仍带自己的动态重定位表，映射时由动态链接器按本次基址改写。

7.12 位置无关代码

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

7.13 库打桩机制

库打桩（library interpositioning）在不改目标函数实现的前提下，把对任意函数的调用接到包装函数：先记账、检查或替换，再视需要调用原实现。拦截可发生在编译时、链接时或加载/运行时，对应名字被改写的层次不同。用途包括沙箱、底层加密、拦截 write 调试，以及 malloc 追踪。课堂例子只调用 malloc(32) 与 free，三种方法都打印地址与大小且不破坏分配语义。

7.13.1 编译时打桩

预处理把调用点的名字换掉。当前目录的 `malloc.h` 含

$$\text{malloc}(size) \mapsto \text{mymalloc}(size), \quad \text{free}(ptr) \mapsto \text{myfree}(ptr).$$

输入是源文本中的记号，输出是替换后的记号；编译调用文件时用 `-I.` 使该头文件覆盖系统头，于是目标文件引用的是 `mymalloc`。包装函数在 `-DCOMPILETIME` 下编译，内部直接调用真正的 `malloc`（编包装时不加 `-I.`，否则递归）。链接器只做普通解析，不需要 `--wrap`。

7.13.2 链接时打桩

调用方目标文件里仍是对 `malloc` 的引用。静态链接器 `ld` 仅在把 `.o` 合成可执行文件时解析符号。`gcc -Wl,--wrap,malloc` 把 `--wrap malloc` 交给 `ld`，规则为

$$\begin{aligned} \text{Resolve}_{ld}(\text{malloc}) &= \text{__wrap_malloc}, \\ \text{Resolve}_{ld}(\text{__real_malloc}) &= \text{malloc}. \end{aligned}$$

左边是目标文件中的符号名，右边是被选中的定义；`Resolveld` 的输出仍是哪一份函数，地址随即按重定位写入可执行文件。包装函数调用 `\_\_real_malloc`，由上述规则接到 `libc`。须重新链接，但不必给调用方源文件加宏。

7.13.3 运行时打桩

包装函数与 `libc` 同名，做成共享库。环境变量 `LD_PRELOAD` 令动态链接器解析未定义引用时先搜该库：

$$\text{Search}(r) = \text{FirstDef}(L_{\text{preload}}, L_{\text{NEEDED}})(r).$$

L_{preload} 是预加载库列表， L_{NEEDED} 是可执行文件的动态依赖；`FirstDef` 返回搜索顺序中第一个定义，故程序中的 `malloc` 先命中包装。包装内

$$\text{mallocp} = \text{dlsym}(\text{RTLD_NEXT}, \text{"malloc"})$$

从当前库之后继续查找，输出 `libc` 中同名函数的地址，经指针调用以避免递归。`free` 同样处理；对空指针直接返回。被拦截程序按普通方式编译，命令行不必出现包装库；不设 `LD_PRELOAD` 则无日志。

7.14 处理目标文件的工具

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

7.15 小结

链接把多个目标文件构造成可执行程序，并可发生在三个时刻。编译时：`ld` 把 `.o` 与静态库成员拷入可执行文件，按 `.rel.*` 填写 `ADDR`；集合 E, U, D 决定抽出哪些成员。加载时：磁盘上的 P 只部分链接，`mmap` 与 `Relocdyn` 在进 `main` 前把 `.so` 焊进进程内存。运行时：`dlopen/dlsym` 按路径与字符串把库映射进已在执行的进程。库打桩在这三层分别用宏、`ld --wrap`、`LD_PRELOAD` 改 `Resolve` 的像，从而截获 `malloc` 等调用。

7.16 练习

原书第 7 章在小结之后为家庭作业，章内另有标号练习题。本节只整理课堂已做的练习题 7.9、家庭作业 7.15，以及 Ubuntu 24.04、gcc 13、默认 PIE 上机测得的数；不补写未做的其余家庭作业。

对可重定位文件 `main.o`，`nm` 在已定义符号旁给出的整数是该符号在所属节内的偏移，不是进程虚拟地址。输入是一个 `.o` 的符号表条目，输出是三元组 $(bind, sect, off)$ ： $bind$ 为 $T/D/U$ 等绑定字母， $sect$ 为 `.text` 或 `.data`， off 为节内字节编号且常从 0 起。 off 的含义是「本模块里从节起点数多少字节」，取值范围是该节长度以内； U 表示本模块只有引用、没有定义，因此没有 off 。函数内自动变量不进入该表。

链接生成 `prog` 后，`objdump -d` 左边一列是链接地址 $ADDR_{link}(s)$ ，即以可执行文件映像起点为 0 的字节编号。默认 PIE 下运行时虚拟地址为

$$ADDR_{run}(s) = B + ADDR_{link}(s),$$

输入是加载基址 B 与链接地址，输出仍是虚拟地址； B 由 ASLR 抽取，故 $ADDR_{run}$ 每次运行可变。上机中 `lea 0x2ed3(%rip),%rdi` 的下一条指令在 `0x113d`，位移字段为小端四字节 `d3 2e 00 00`，即

$$disp = 0x2ed3, \quad ADDR_{link}(array) = 0x113d + 0x2ed3 = 0x4010.$$

左边 `disp` 是有符号字节位移，不是地址；右边 `0x4010` 是 `array` 的链接地址。该处类型为 `R_X86_64_PC32`，指令里保存的是 `disp` 而非把 `0x4010` 整段写入立即数。

对 `call`，位移 `0x5` 满足

$$(call \text{ 的下一条地址}) + 0x5 = ADDR_{link}(sum),$$

故 `0x5` 同样不是 `sum` 的绝对地址。

静态库 `libvector.a` 用 $|ar(-t)|$ 计数成员：输入是归档，输出是其中 `.o` 的个数。该数是打包进去的模块个数，不是运行时调用次数。链接器只抽出当时未定义集合 U 所需

要的成员，故库中可同时有 `addvec.o` 与 `multvec.o`，而 `prog2c` 只含前者。命令行从左到右扫描：库出现在引用它的 `.o` 之前则扫描时 $U = \emptyset$ ，报 `undefined reference`。

运行时链接的输入是路径字符串 p 与符号名字符串 n ，

$$h = dlopen(p, RTLD_LAZY), \quad a = dlsym(h, n),$$

输出 h 为句柄、 a 为本次映射中的函数地址。可执行文件因此不必在链接命令行出现 `libvector.so`，通常只需 `-ldl`。

练习题 7.9 的两个模块中，`foo6.c` 的 `int main()` 是强符号，`bar6.c` 的 `char main`；在旧规则下是 COMMON 弱符号。成立条件为「一个强符号与若干弱符号同名则选强符号」，于是

$$\text{Resolve}(\text{main}_{\text{var}}) = \text{DEF}(\text{main}_{\text{func}}).$$

输入是两个同名定义，输出是唯一被选中的那一份实体，即函数 `main` 的机器码，而不是另开的一字节变量。p2 按 `char` 读取该对象，得到的是

$$b = M_8[\text{ADDR}(\text{main})], \quad v = \text{SignExt}_{8 \rightarrow 32}(b).$$

b 是函数首字节， v 是 `printf("%x")` 打印的整数。若 $b < 0x80$ 则 $v = b$ ；若 $b \geq 0x80$ 则 v 的高位全 1。教材环境 `main` 常以 `push %rbp` 开头，其操作码 $b = 0x55$ ，故 $v = 0x55$ 。`0x55` 只是该指令的编码，不是 `main` 必须具备的标记；p2 自身也可以含 `push %rbp`，与打印值无关。上机中 `main` 在 `0x1149` 起为 `endbr64(f3 0f 1e fa)`，故 $b = 0xf3$ 、 $v = 0xfffffffff3$ ；`push %rbp` 排在 `0x114d`，不是被读的那一字节。p2 中 `movzbl -0x23(%rip)` 的下一条 RIP 为 `0x116c`，有效地址

$$0x116c + (-0x23) = 0x1149 = \text{ADDR}(\text{main}),$$

与反汇编中 `<main>` 起点一致。GCC 10 起默认 `-fno-common`，未初始化的 `char main`；也是强符号，两个强 `main` 使链接报 `multiple definition`；要复现教材的弱符号合并需显式 `-fcommon`。

家庭作业 7.15 在本机测得：`libc.a` 含 2211 个目标文件。`libm.a` 不是 `ar` 归档而是链接脚本

$$\text{GROUP}(\text{libm-2.39.a}, \text{libmvec.a}),$$

二者分别含 801 与 548 个 `.o`。这些整数是归档成员个数，不是某次调用的耗时。

对同一源以 `-O2` 与 `-O2 -g` 各生成一份可执行文件，测得体积 16024 与 17984 字节，`cmp` 自文件前部即不同；`objdump -d` 的 `.text` 除文件名行外相同。故 `-g` 的输入是已优化的指令流，输出是附加的 `.debug_*` 节；体积差来自调试信息，不是 `main` 被改写成另一套算法。

`ldd` 列出 `gcc` 驱动程序为该程序记录的共享库。`libc.so.6` 提供 `printf` 等 C 库函数；`ld-linux-x86-64.so.2` 是动态链接器，加载时把各 `.so` 映射进进程并完成动态重定

位；`linux-vdso.so.1` 由内核映射，不对应磁盘文件，用于加速部分系统调用。三者的输出都是「这份进程映像依赖哪些动态对象」，不是静态库成员计数。

第八章 异常控制流

异常控制流 (exceptional control flow, ECF) 是对处理器状态变化的响应：控制不再按下一条指令前进，而是转入操作系统内核或另一个进程。它实现部分在硬件、部分在操作系统。本章对应课堂讲义 14-ecf-procs (异常、进程与进程控制)；信号、非本地跳转与相关工具尚未讲授，对应各节仍只保留目录。

8.1 异常

异常 (exception) 是对事件 (event, 处理器状态的变化) 的响应：控制从应用程序突然转到操作系统内核 (kernel)。内核是操作系统中常驻内存的那一部分，不是 `top` 里与用户程序并列的一个进程。事件可以是当前指令直接引起的 (除零、算术溢出、缺页)，也可以与当前指令无关 (定时器到期、I/O 完成、键入 Ctrl-C)。

Exceptions

- An **exception** is a transfer of control to the OS *kernel* in response to some *event* (i.e., change in processor state)
 - Kernel is the memory-resident part of the OS
 - Examples of events: Divide by 0, arithmetic overflow, page fault, I/O request completes, typing Ctrl-C

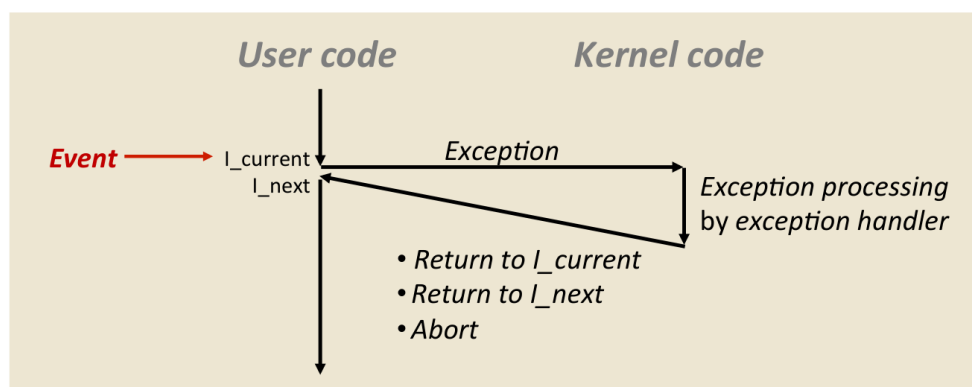


图 8.1 异常的剖析：事件把控制从应用程序转到异常处理程序。

符号: I_{current} : 发生事件时正在执行的指令; I_{next} : 若无异常则将执行的下一条; handler: 内核中处理该事件的子程序

处理程序结束后三条路: 回到 I_{current} 、回到 I_{next} , 或中止被中断的程序。输入是当前指令与事件, 输出是这三种控制转移之一。

8.1.1 异常处理

系统启动时, 操作系统分配并初始化一张跳转表, 称为异常表 (exception table), 又称中断向量。表项 k 存放异常号为 k 的处理程序入口地址。运行时处理器检测到事件, 确定异常号 k , 经异常表做间接调用。

Exception Tables

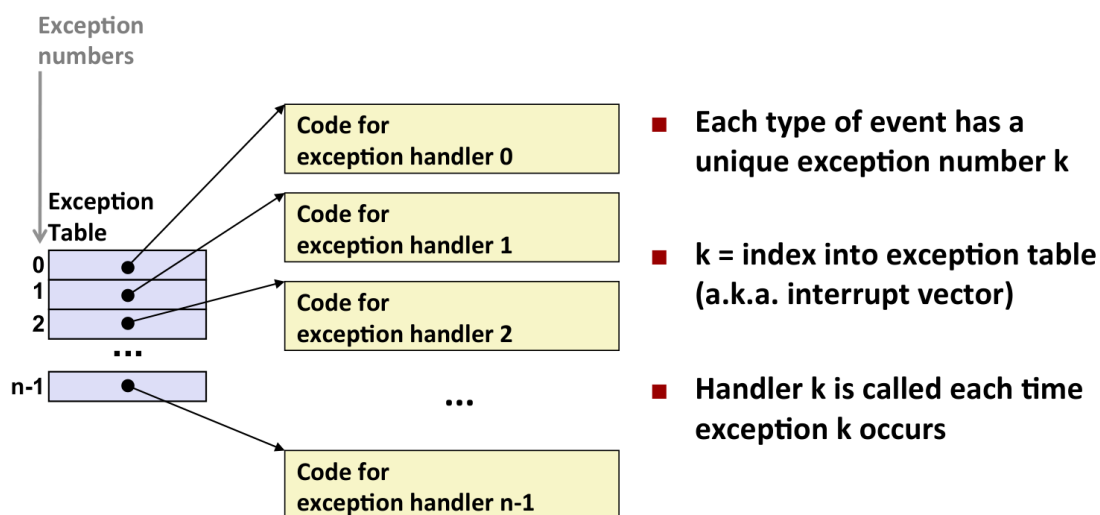


图 8.2 异常表: 第 k 项是异常 k 的处理程序地址。

符号: k : 异常号, 非负整数下标; handler k : 对应内核子程序的入口

$$\text{Handler}(k) = \text{Table}[k], \quad k \in \{0, 1, \dots, n-1\}.$$

输入 k 由硬件提供: 同步异常由 CPU 根据指令译码与执行结果生成, 异步中断由中断控制器放到总线上。输出 $\text{Handler}(k)$ 是内核代码的虚拟地址, 不是用户程序里的函数指针。 k 的取值范围是该体系结构异常表的合法下标, 例如 x86 上缺页、除零各有固定编号。

8.1.2 异常的分类

异常分为四类。图 8.4 的属性写成

类别	起因	异步/同步	返回行为
中断	I/O 设备信号	异步	总是回到 I_{next}
陷阱	有意的异常	同步	总是回到 I_{next}
故障	可能恢复的错误	同步	可能回到 I_{current}
终止	不可恢复的错误	同步	永不返回

图 8.4 异常的分类。

符号：同步 = 由正在执行的指令直接引起；异步 = 来自处理器外部。

中断（interrupt）由外部设备拉高中断脚并提供异常号。定时器中断使内核能从用户程序收回 CPU，用于调度。处理完回到 I_{next} 。

陷阱（trap）是有意执行的指令，例如系统调用与断点。处理完回到 I_{next} ，对用户像一次进入内核的过程调用。

故障（fault）由当前指令无意触发，但内核或许能补救。缺页常把那一页调入物理内存后重做 I_{current} ；保护错往往不可恢复，处理程序中止进程。浮点异常也属此类。

终止（abort）由不可恢复错误引起，例如硬件校验错。处理程序中止当前程序，不回到用户代码。

8.1.3 Linux/x86-64 系统中的异常

x86-64 Linux 上，故障与陷阱的例子包括：除零、缺页、通用保护、以及 `syscall` 指令引起的系统调用。系统调用是陷阱：用户程序把编号放入约定寄存器再执行 `syscall`。

编号	名字	作用
0	read	读文件
1	write	写文件
2	open	打开文件
3	close	关闭文件
4	stat	取文件信息
57	fork	创建进程
59	execve	执行程序
60	_exit	结束进程
62	kill	向进程发信号

图 8.10 Linux x86-64 上若干系统调用。

符号：编号 = 进入内核时 `%rax` 的值；名字 = C 库包装函数。

编号是内核约定，与缺页等异常号不是同一张日常清单。C 中写 `open`、`fork`，最终落到这些编号之一。

- User calls: `open(filename, options)`
- Calls `__open` function, which invokes system call instruction `syscall`

```

0000000000e5d70 <__open>:
...
e5d79:  b8 02 00 00 00      mov  $0x2,%eax  # open is syscall #2
e5d7e:  0f 05               syscall          # Return value in %rax
e5d80:  48 3d 01 f0 ff ff    cmp  $0xffffffffffff001,%rax
...
e5dfa:  c3                 retq

```

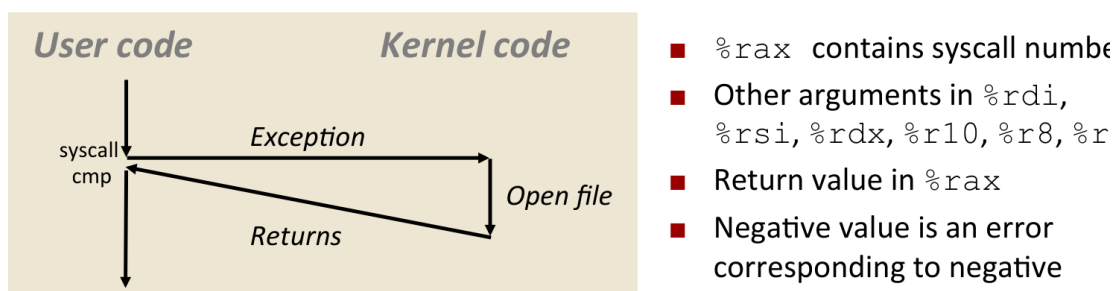


图 8.6 系统调用示例：`open` 经 `syscall` 进内核，返回后继续下一条。

符号：`%rax`：进去时为调用号 2，回来时为返回值； I_{next} ：`syscall` 之后的比较指令

x86-64 Linux 约定：进入时 `%rax` 为调用号，`%rdi`、`%rsi`、`%rdx`、`%r10`、`%r8`、`%r9` 为参数；返回后 `%rax` 为结果，负值表示出错。输入是调用号与参数寄存器，输出是返回值标量。该标量对 `open` 是文件描述符或错误码的相反数一类约定。

Fault Example: Page Fault

- User writes to memory location
- That portion (page) of user's memory is currently on disk

```
int a[1000];
main ()
{
    a[500] = 13;
}
```

```
80483b7:    c7 05 10 9d 04 08 0d    movl    $0xd,0x8049d10
```

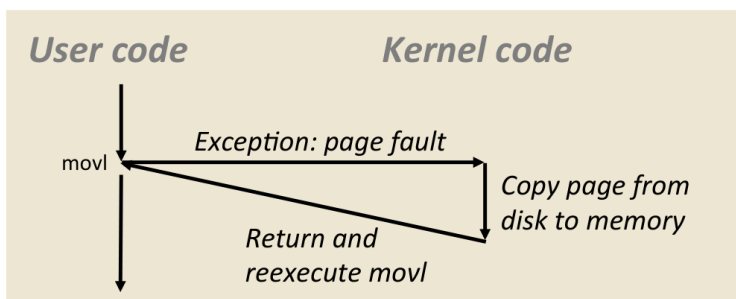


图 8.7 故障示例：合法地址缺页时拷页并重做 `movl`。

符号： I_{current} ：引起缺页的写指令；`disk→mem`：内核把该页调入物理内存

对 `a[500]=13`，虚拟地址合法但页不在内存：同步故障。内核拷页后回到 I_{current} 重做同一条 `movl`。对 `a[5000]=13`，硬件仍先报缺页，内核发现地址无合法映射，向该进程发 `SIGSEGV`，默认终止，不再重做。`SIGSEGV` 是段错误对应的信号编号（Linux 上常为 11），表示当前进程访问了不该访问的地址或权限不够。

8.2 进程

进程（process）是一个正在运行的程序的实例（instance）。它不是磁盘上的可执行文件，也不是处理器硬件。同一份程序可以同时对应多个进程；一个核可以轮流跑多个进程。

Processes

- **Definition: A *process* is an instance of a running program.**
 - One of the most profound ideas in computer science
 - Not the same as “program” or “processor”
- **Process provides each program with two key abstractions:**
 - ***Logical control flow***
 - Each program seems to have exclusive use of the CPU
 - Provided by kernel mechanism called *context switching*
 - ***Private address space***
 - Each program seems to have exclusive use of main memory.
 - Provided by kernel mechanism called *virtual memory*

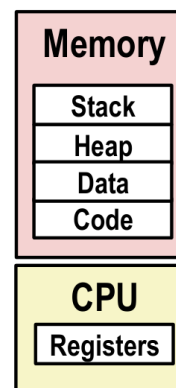


图 8.13 进程提供逻辑控制流与私有地址空间两层抽象。

符号：CPU/Registers：该进程眼里的处理器；Stack/Heap/Data/Code：该进程的虚拟存储器分区

进程给每个程序两层关键抽象：逻辑控制流（好像独占 CPU）与私有地址空间（好像独占主存）。前者由上下文切换提供，后者由虚拟内存提供。

8.2.1 逻辑控制流

处理器从开机到关机执行的指令序列是物理控制流。每个进程对应其中一段交错后的投影，称为逻辑控制流：进程自己看见的指令一条接一条，中间插入的别的进程被抽象掉。上下文切换把当前进程的寄存器等保存到内存，装入另一个进程的现场，使每个程序以为 CPU 只在执行自己。

8.2.2 并发流

两个进程并发（concurrent），当且仅当它们的逻辑流在时间上重叠；否则称为顺序（sequential）。并发不要求同一时刻两条用户指令都在占用同一颗核。

Concurrent Processes

- Each process is a logical control flow.
- Two processes *run concurrently* (are concurrent) if their flows overlap in time
- Otherwise, they are *sequential*
- Examples (running on single core):
 - Concurrent: A & B, A & C
 - Sequential: B & C

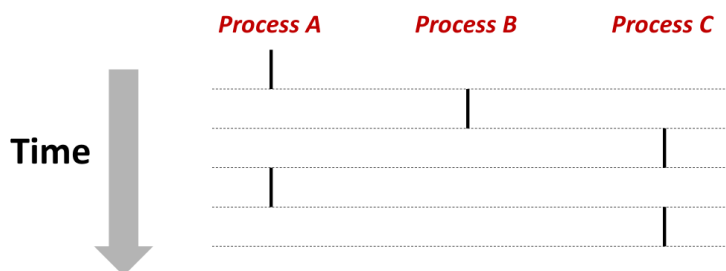


图 8.12 单核上三个进程的交错：流的生存期重叠则并发。

符号：Time：向下为时间；竖杠：该时间片内核在执行哪个进程

设进程 A, B, C 的生存区间为 $[\alpha_X, \omega_X]$ 。

$$\text{Concurrent}(X, Y) \iff [\alpha_X, \omega_X] \cap [\alpha_Y, \omega_Y] \neq \emptyset.$$

输入是两条流的起止时刻，输出是真假。真表示二者并发，假表示顺序。单核示例中 A 与 B 、 A 与 C 重叠故并发， B 在 C 开始前结束故顺序。

物理上，单核并发流的执行片段在时间上不相交；用户视角仍可把各流想成彼此并行。多核时每个核可同时执行一个不同进程，调度仍由内核完成。

8.2.3 私有地址空间

每个进程有自己的虚拟地址空间：代码、数据、堆、栈。相同的虚拟地址在不同进程中映射到不同的页。这解释了缺页非法时只终止当前 PID，以及 `fork` 之后父子对 `x` 的修改互不影响。

8.2.4 用户模式和内核模式

处理器用模式位区分用户模式与内核模式。用户模式不能执行特权指令、不能直接改页表或访问设备。异常把控制转入内核并把模式改为内核模式；处理程序作为某个已存在进程的一部分运行，内核不是单独的进程。从内核返回用户代码时恢复用户模式。

8.2.5 上下文切换

Context Switching

- **Processes are managed by a shared chunk of memory-resident OS code called the *kernel***
 - Important: the kernel is not a separate process, but rather runs as part of some existing process.
- **Control flow passes from one process to another via a *context switch***

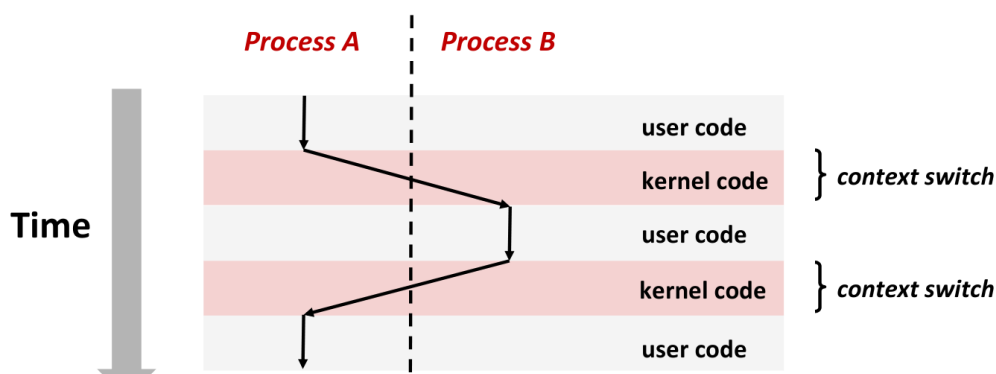


图 8.14 控制流在内核中从进程 A 转到进程 B。

符号：user code：用户指令；kernel code：内核中保存/装入寄存器并切换地址空间的那段

一次上下文切换包含：把当前寄存器写入该进程的保存区，调度下一个进程，装入其保存的寄存器并切换地址空间（换页表）。切换发生在内核中，由系统调用、缺页或定时器中断等异常触发。单核上任意时刻只有一套活寄存器；未执行进程的寄存器值存在内存里。

8.3 系统调用错误处理

Linux 系统级函数失败时通常返回 -1 ，并设置全局变量 `errno` 表示原因。必须检查每个非 `void` 系统级函数的返回值。

$$\text{Fail}(f) \iff r = -1,$$

此时 `strerror(errno)` 把编号译成字符串。 r 是函数返回的标量； -1 表示失败，非负（或约定的成功值）表示成功。

课堂把打印与退出收成

```
void unix_error(char *msg)
```

```
{
    fprintf(stderr, "%s: %s\n", msg, strerror(errno));
    exit(0);
}
```

再把调用与检查包成 Stevens 风格包装：大写 Fork 内调 `fork`，失败则 `unix_error`，成功才把 `pid` 返回给调用者。小写名为系统函数，大写名为课件包装，不是内核另提供的系统调用。

8.4 进程控制

程序员通过系统调用创建、终止、回收进程并加载新程序。

8.4.1 获取进程 ID

$$p = \text{getpid}(), \quad p_{\text{par}} = \text{getppid}().$$

输入为空，输出是 `pid_t` 整数。 p 是当前进程的 PID， p_{par} 是父进程的 PID，取值在内核分配的正整数编号中。PID 0 不是用户进程编号。

8.4.2 创建和终止进程

从程序员视角，进程处于三态之一。Running：正在执行，或等待被调度且终将被内核选中（含等待 I/O）。Stopped：挂起，在收到继续通知前不会被调度。Terminated：永久停止。

进程因下列原因之一变为 Terminated：收到默认动作为终止的信号（如 SIGSEGV）、从 `main` 返回、或调用 `exit`。

$$\text{exit}(status)$$

结束当前进程并留下退出码 `status`。约定 0 为正常，非 0 为出错。从 `main` 返回整数 n 等价于 `exit(n)`。`exit` 调用一次、永不返回。

父进程调用 `fork` 创建新的 Running 子进程。

$$r = \text{fork}().$$

输入为空。输出 r 在两个进程中各出现一次：父进程中 r 等于孩子的 PID（正整数），子进程中 $r = 0$ ，失败时 $r = -1$ 。0 只出现在孩子对返回值的使用中，不是孩子的 PID；孩子的真实编号由 `getpid` 给出。

孩子得到父进程虚拟地址空间的一份相同但分离的拷贝，以及打开文件描述符的拷贝，并具有不同的 PID。fork 调用一次、返回两次。

```
int x = 1;
pid = Fork();
if (pid == 0) {          /* Child */
    printf("child : x=%d\n", ++x);
    exit(0);
}
printf("parent: x=%d\n", --x);
exit(0);
```

Fork 返回时两份 x 都是 1。孩子执行 $++x$ 印 2，父亲执行 $--x$ 印 0，之后各自独立。谁先 printf 不能事先保证。孩子必须 exit，否则会落到父亲的打印语句。两边的 stdout 常仍指向同一终端。

进程图（process graph）刻画并发语句的偏序：顶点是一次语句执行，边 $a \rightarrow b$ 表示 a 一定发生在 b 之前。任意拓扑序对应一种可行的总顺序。

Process Graph Example

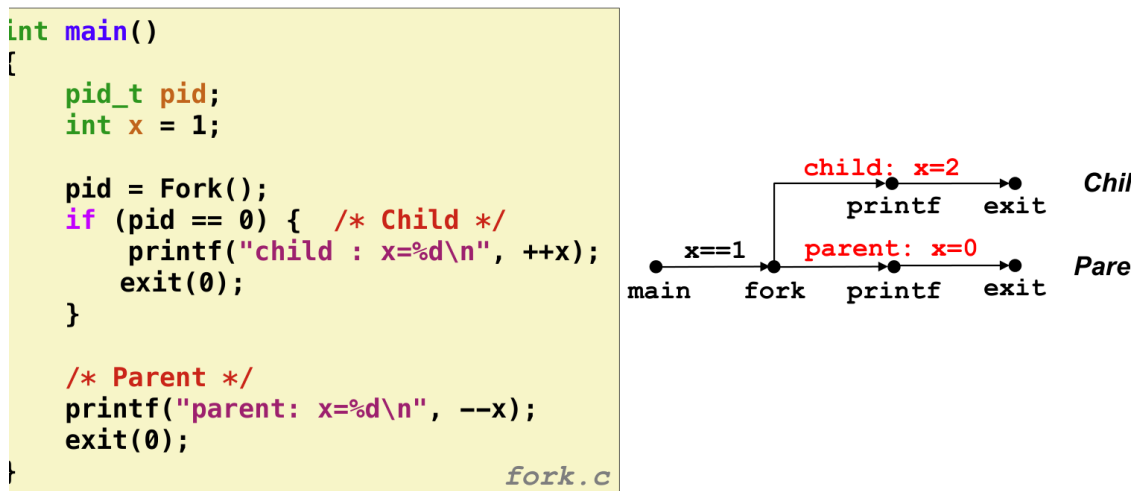


图 8.16 fork.c 的进程图：fork 处分叉，两支 printf 之间无箭头。

符号： $x=1$ ：分叉前的变量；child $x=2$ / parent $x=0$ ：两支上的输出

连续两次无 if 的 fork 使每个进程都继续并再分叉：L0 印 1 次，L1 印 2 次，Bye 印 4 次。每个 Bye 顶点都在该进程自己的 L1 之后，故全局第一个 Bye 前至少有一个 L1。

`if (fork() != 0)` 只让父进程继续嵌套，得到 3 个进程，L0、L1、L2 各 1 次，Bye 共 3 次；最初父进程的路径含 L2→Bye。`if (fork() == 0)` 只让子进程继续嵌套，次数相同，链在孩子一侧；孙子的路径含 L2→Bye。

8.4.3 回收子进程

进程终止后仍占用退出码与内核表项，称为僵尸（zombie）。父进程用 `wait` 或 `waitpid` 回收（reap）：

$$w = \text{wait}(\&\text{status}).$$

输入是指向整数的指针（或 NULL）。输出 w 是被回收孩子的 PID。若指针非空，则 status 被写成编码后的结束原因。用 `<sys/wait.h>` 中的宏解码：`WIFEXITED` 为真时 `WEXITSTATUS` 给出 `exit` 的整数；`WIFSIGNALED` 为真时 `WTERMSIG` 给出信号编号（如 `SIGSEGV`）。这些宏的输入是 status 那一个整数，输出是真假或非负编号。

`wait` 挂起当前进程直到它的某一个孩子终止；多个已结束的孩子按任意顺序回收。一次调用回收一个孩子，成功 `fork` 了 N 个就要回收 N 次。

$$w = \text{waitpid}(\text{pid}, \&\text{status}, \text{opt}).$$

$\text{pid} > 0$ 只等该进程； $\text{pid} = -1$ 与 `wait` 等价。 $\text{opt} = 0$ 表示死等结束；`WNOHANG` 使无状态可报时立即返回 0；`WUNTRACED`、`WCONTINUED` 分别在停止与继续时也可返回。 w 为孩子 PID、在 `WNOHANG` 下可为 0，出错为 -1 。

父进程不回收且自己也终止时，孤儿由 `init`（PID 1）代收。长命进程（shell、服务器）必须显式回收，否则僵尸占满进程表。孩子已 `exit`、父亲死循环不 `wait` 时，`ps` 显示 `<defunct>`；杀死父亲后 `init` 收掉该行。父亲先 `exit`、孩子死循环时，孩子是仍在运行的孤儿，不是僵尸，须另外结束该 PID。

`wait` 在进程图上增加孩子 `exit` 到父亲 `wait` 的边，因此 CT（child terminated）不能出现在孩子的打印之前。

8.4.4 让进程休眠

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

8.4.5 加载并运行程序

`execve` 在当前进程中加载并运行一个新程序：

$$r = \text{execve}(\text{filename}, \text{argv}, \text{envp}).$$

输入 *filename* 是可执行文件或以 `#!` 开头的脚本路径, *argv*、*envp* 是以 `NULL` 结尾的字符串指针数组, 约定 *argv*[0] 为程序名。成功时不返回 (*r* 不出现); 失败返回 `-1`。它覆盖代码、数据与栈, 保留 PID、打开的文件与部分信号上下文。旧用户地址空间被取消映射, 内核在高地址重新铺用户栈且仍向低地址增长。

Structure of the stack when a new program starts

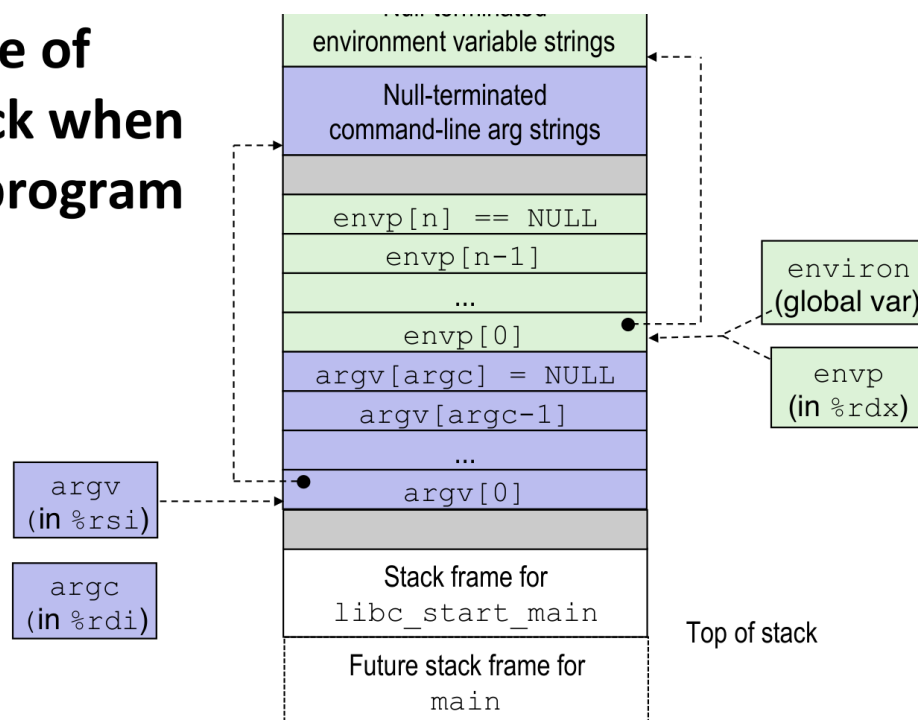


图 8.22 新程序启动时用户栈的组织。

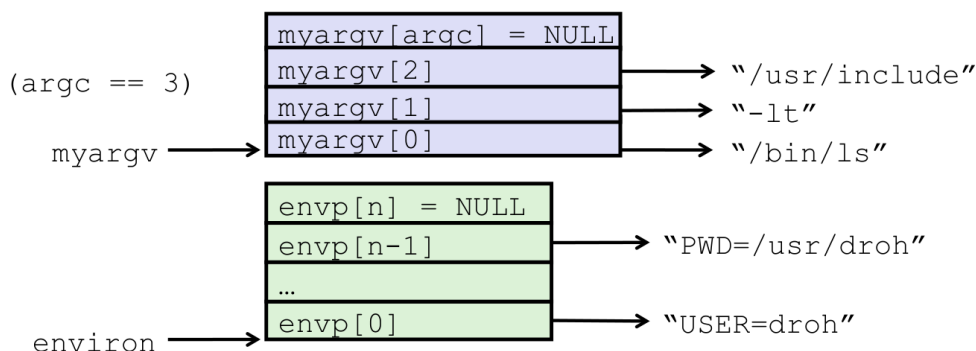
符号: `argc` 在 `%rdi`; `argv` 在 `%rsi`; `envp` 在 `%rdx`; `environ`: 指向环境表的全局指针

高地址存放环境串与参数串, 其下为 `envp[]`、`argv[]`, 再下为 `libc_start_main` 与将来 `main` 的栈帧。x86-64 把 `argc`、`argv`、`envp` 放入 `%rdi`、`%rsi`、`%rdx` 后调用 `main`。`argc` 是参数个数 (不含结尾 `NULL`), 为非负整数。

8.4.6 利用 fork 和 execve 运行程序

execve Example

- Executes `"/bin/ls -lt /usr/include"` in child process using current environment:



```
if ((pid = Fork()) == 0) { /* Child runs program */
    if (execve(myargv[0], myargv, environ) < 0) {
        printf("%s: Command not found.\n", myargv[0]);
        exit(1);
    }
}
```

图 8.20 孩子中 `execve("/bin/ls", myargv, environ)` 的参数与环境表。

符号: `myargv[0..2]`: `/bin/ls`、`-lt`、`/usr/include`; `argc=3`

典型用法: 父亲 `fork`, 孩子调用 `execve`, 父亲 `wait`。

```
if ((pid = Fork()) == 0) {
    if (execve(myargv[0], myargv, environ) < 0) {
        printf("%s: Command not found.\n",
            myargv[0]);
        exit(1);
    }
}
```

成功则该 PID 的映像换成 `ls`, 旧代码不回到 `printf`。失败才印错误并以 1 退出。`ls` 是已经链接好的可执行文件, 不必与当前程序静态链在一起。父亲仍运行原来的程序并回收孩子; 没有「把旧程序再装回该 PID」这一步。

8.5 信号

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充, 此处不编造结论。

8.5.1 信号术语

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

8.5.2 发送信号

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

8.5.3 接收信号

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

8.5.4 阻塞和解除阻塞信号

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

8.5.5 编写信号处理程序

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

8.5.6 同步流以避免讨厌的并发错误

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

8.5.7 显式地等待信号

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

8.6 非本地跳转

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

8.7 操作进程的工具

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

8.8 小结

异常是需要非标准控制流的事件，由外部中断或内部陷阱、故障（及终止）产生。进程在系统中同时存在多个；单核同一时刻只执行一个，但每个进程都表现为独占处理器与私有地址空间。创建进程调用 `fork`：一次调用、两次返回。结束进程调用 `exit`：一次调用、不返回。回收与等待调用 `wait` 或 `waitpid`。加载并运行新程序调用 `execve`：一次调用，成功则不返回。

第九章 虚拟内存

本章对应原书第 9 章 Virtual Memory。课堂尚未讲授，以下仅保留目录。

9.1 物理和虚拟寻址

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.2 地址空间

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.3 虚拟内存作为缓存的工具

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.3.1 DRAM 缓存的组织

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.3.2 页表

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.3.3 页命中

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.3.4 缺页

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.3.5 分配页面

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.3.6 再论局部性

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.4 虚拟内存作为内存管理的工具

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.5 虚拟内存作为内存保护的工具

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.6 地址翻译

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.6.1 结合缓存和虚拟内存

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.6.2 利用 TLB 加速地址翻译

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.6.3 多级页表

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.6.4 端到端的地址翻译

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.7 案例研究：Intel Core i7/Linux 内存系统

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.7.1 Core i7 地址翻译

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.7.2 Linux 虚拟内存系统

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.8 内存映射

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.8.1 再看共享对象

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.8.2 再看 fork 函数

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.8.3 再看 execve 函数

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.8.4 使用 mmap 函数的用户级内存映射

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.9 动态内存分配

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.9.1 malloc 和 free 函数

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.9.2 为什么要使用动态内存分配

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.9.3 分配器的要求和目标

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.9.4 碎片

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.9.5 实现问题

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.9.6 隐式空闲链表

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.9.7 放置已分配块

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.9.8 分割空闲块

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.9.9 获取额外的堆内存

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.9.10 合并空闲块

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.9.11 带边界标记的合并

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.9.12 实现一个简单的分配器

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.9.13 显式空闲链表

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.9.14 分离的空闲链表

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.10 垃圾收集

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.10.1 垃圾收集器的基本知识

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.10.2 Mark&Sweep 垃圾收集器

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.10.3 C 程序的保守 Mark&Sweep

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.11 C 程序中常见的与内存有关的错误

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.11.1 间接引用坏指针

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.11.2 读未初始化的内存

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.11.3 允许栈缓冲区溢出

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.11.4 假设指针和它们指向的对象是相同大小的

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.11.5 造成错位错误

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.11.6 引用指针，而不是它所指向的对象

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.11.7 误解指针运算

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.11.8 引用不存在的变量

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.11.9 引用空闲堆块中的数据

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.11.10 引起内存泄漏

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

9.12 小结

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

第十章 系统级 I/O

本章对应原书第 10 章 System-Level I/O。课堂尚未讲授，以下仅保留目录。

10.1 Unix I/O

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

10.2 文件

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

10.3 打开和关闭文件

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

10.4 读和写文件

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

10.5 用 Rio 包健壮地读写

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

10.5.1 Rio 无缓冲的输入输出函数

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

10.5.2 Rio 带缓冲的输入函数

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

10.6 读取文件元数据

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

10.7 读取目录内容

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

10.8 共享文件

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

10.9 I/O 重定向

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

10.10 标准 I/O

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

10.11 综合：该使用哪些 I/O 函数

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

10.12 小结

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

第十一章 网络编程

本章对应原书第 11 章 Network Programming。课堂尚未讲授，以下仅保留目录。

11.1 客户端—服务器编程模型

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

11.2 网络

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

11.3 全球 IP 因特网

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

11.3.1 IP 地址

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

11.3.2 因特网域名

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

11.3.3 因特网连接

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

11.4 套接字接口

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

11.4.1 套接字地址结构

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

11.4.2 socket 函数

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

11.4.3 connect 函数

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

11.4.4 bind 函数

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

11.4.5 listen 函数

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

11.4.6 accept 函数

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

11.4.7 主机和服务的转换

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

11.4.8 套接字接口的辅助函数

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

11.4.9 示例回声客户端和服务端

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

11.5 Web 服务器

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

11.5.1 Web 基础

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

11.5.2 Web 内容

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

11.5.3 HTTP 事务

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

11.5.4 服务动态内容

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

11.6 综合：TINY Web 服务器

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

11.7 小结

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

第十二章 并发编程

本章对应原书第 12 章 Concurrent Programming。课堂尚未讲授，以下仅保留目录。

12.1 基于进程的并发编程

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.1.1 基于进程的并发服务器

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.1.2 进程的优劣

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.2 基于 I/O 多路复用的并发编程

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.2.1 基于 I/O 多路复用的并发事件驱动服务器

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.2.2 I/O 多路复用的优劣

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.3 基于线程的并发编程

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.3.1 线程执行模型

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.3.2 Posix 线程

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.3.3 创建线程

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.3.4 终止线程

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.3.5 回收已终止的线程

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.3.6 分离线程

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.3.7 初始化线程

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.3.8 基于线程的并发服务器

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.4 多线程程序中的共享变量

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.4.1 线程内存模型

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.4.2 将变量映射到内存

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.4.3 共享变量

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.5 用信号量同步线程

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.5.1 进度图

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.5.2 信号量

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.5.3 使用信号量来实现互斥

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.5.4 利用信号量来调度共享资源

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.5.5 综合：基于预线程化的并发服务器

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.6 使用线程提高并行性

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.7 其他并发问题

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.7.1 线程安全

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.7.2 可重入性

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.7.3 在线程化的程序中使用已存在的库函数

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.7.4 竞争

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.7.5 死锁

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。

12.8 小结

本节标题依 *Computer Systems: A Programmer's Perspective* 第三版目录保留。正文待后续课堂补充，此处不编造结论。